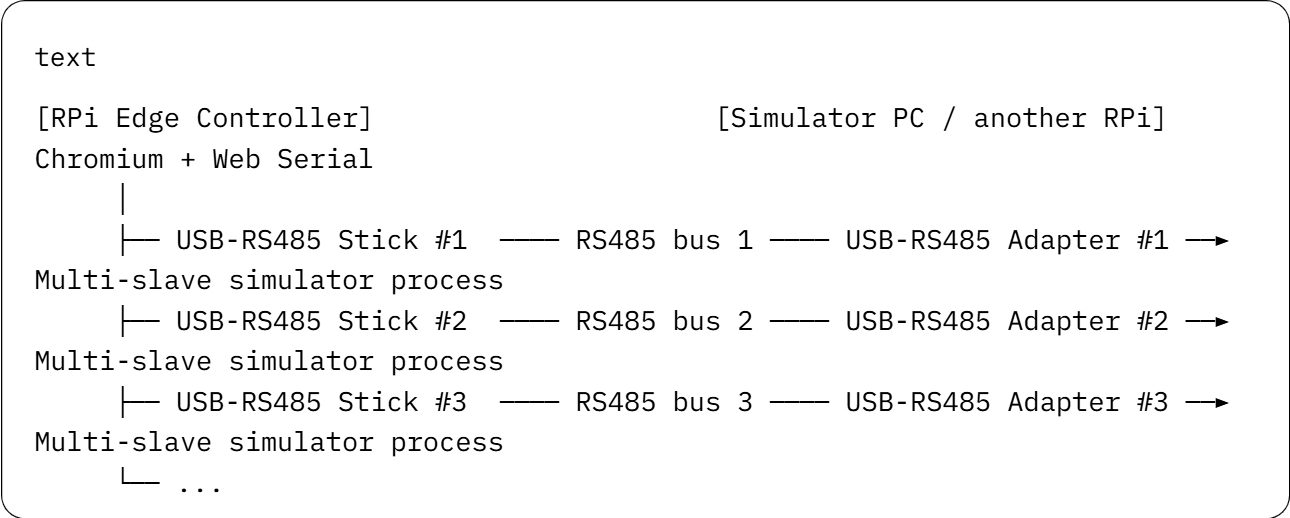


Yes — the real RS485 approach is the right choice if you want partners/clients to see a setup that looks and behaves like a real multi-bus installation.

Recommended Architecture



- Each physical USB-RS485 stick on the RPi stays as a real serial port with its real `usbVendorId`.
- Chromium opens them via Web Serial exactly as it would in production.
- On the other end of each RS485 cable you put a matching USB-RS485 adapter connected to a simulator machine.
- One simulator process per bus can emulate many devices (different slave IDs).

This is the most convincing demo: real USB devices, real RS485 electrical layer, real multi-drop behaviour, real timing.

What you need

Hardware (simulator side)

- One PC or another Raspberry Pi
- One USB-RS485 adapter **per bus** (same type you already use is fine)
- RS485 cable + proper termination (120 Ω at both ends of each bus is best practice)
- Optional: bias resistors if the adapters don't provide them

Software (simulator side) – good options that support multiple slaves on one serial port:

Tool	Platform	Multi-slave	Notes

pymodbus	Linux / Windows	Excellent	Free, very flexible, recommended
modbus_slave (lab240)	Linux / Windows	Up to 30 slaves	Lightweight CLI, good for quick demos
QSlave	Linux / Windows	Yes	GUI, designed for physical RS485 networks
ModRSSim2	Windows	Yes	Popular free GUI simulator
diagslave	Linux / Windows	One ID per instance	Simple, run multiple processes if needed
Commercial (Witte Modbus Slave, etc.)	Windows	Yes	Polished GUI if you prefer

Practical tips for a clean demo

1. One process = one bus

Each simulator process opens one serial port and answers multiple slave IDs (1, 2, 3...).

2. Match the real parameters

Use the same baud rate, parity, stop bits that your production devices use (commonly 9600 8N1 or 19200 8E1).

3. Termination & wiring

Put 120 Ω termination at both ends of every bus. Keep cable runs short for demos (a few metres is fine).

4. Make values change

Use scripts or the simulator's built-in random/file/update features so registers actually move. Static values look less real.

5. Label everything

Clearly mark which USB stick / which bus / which slave IDs belong together. Clients notice this.

6. Optional but nice

Put the simulator machine on a small DIN-rail or in a box next to the RPi so the whole setup looks like a proper test bench.

Minimal starting setup (3 buses)

- 3× USB-RS485 sticks already on the RPi
- 3× USB-RS485 adapters on a laptop/PC

- 3 short RS485 cables
- One pymodbus (or modbus_slave) process per bus

Would you like a ready-to-run pymodbus multi-slave example for one bus, or a systemd service template so the simulators start automatically on the second machine?